

**LAPORAN PRAKTIKUM**  
**ALGORITMA DAN PEMROGRAMAN 2**  
**MODUL 18**  
**“MESIN ABTRAK”**



**DISUSUN OLEH:**  
**Alvin Setya Wardana**  
**109082500107**  
**S1 IF-13-02 DOSEN:**  
**WAHYU ANDI SAPUTRA**

**PROGRAM STUDI S1 TEKNIK INFORMATIKA**  
**FAKULTAS INFORMATIKA**  
**TELKOM UNIVERSITY PURWOKERTO**

## DASAR TEORI

### Dasar Teori – Mesin Abstrak Karakter

Mesin abstrak karakter adalah model konseptual yang digunakan untuk membaca dan memproses rangkaian karakter secara sistematis. Mesin ini bekerja dengan prinsip **automata sederhana**, di mana terdapat pointer yang menunjuk ke posisi karakter saat ini, kemudian digerakkan maju hingga mencapai tanda akhir (penanda titik .).

Operasi dasar yang digunakan meliputi:

- **start()** untuk memulai pembacaan dari karakter pertama.
- **maju()** untuk menggeser posisi pembacaan ke karakter berikutnya.
- **eop()** untuk memeriksa apakah pembacaan sudah mencapai akhir string.
- **cc()** untuk mengambil karakter yang sedang dibaca.

Dengan operasi tersebut, mesin abstrak karakter dapat digunakan untuk berbagai analisis string, seperti menghitung jumlah karakter, menghitung frekuensi huruf tertentu, serta mendeteksi pola kata tertentu (misalnya pasangan huruf "LE"). Konsep ini penting karena melatih pemahaman dasar tentang **pemrosesan data sekuensial** dan menjadi pondasi bagi algoritma yang lebih kompleks dalam bidang pemrograman dan teori bahasa formal.

## 1. Latihan1 Source

Code:

```
package main

import (
    "fmt"
)

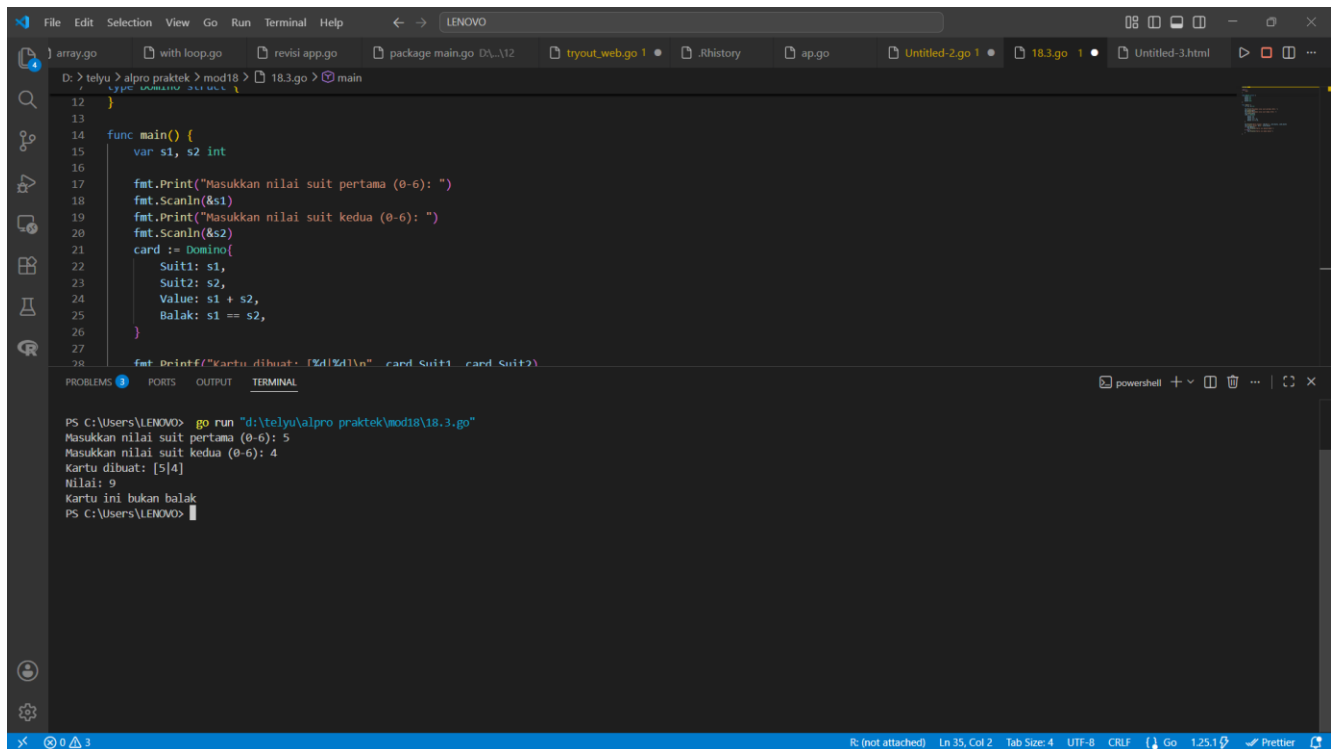
type Domino struct {
    Suit1 int
    Suit2 int
    Value int
    Balak bool
}

func main() {
    var s1, s2 int

    fmt.Print("Masukkan nilai suit pertama (0-6): ")
    fmt.Scanln(&s1)
    fmt.Print("Masukkan nilai suit kedua (0-6): ")
    fmt.Scanln(&s2)
    card := Domino{
        Suit1: s1,
        Suit2: s2,
        Value: s1 + s2,
        Balak: s1 == s2,
    }

    fmt.Printf("Kartu dibuat: [%d|%d]\n", card.Suit1, card.Suit2)
    fmt.Printf("Nilai: %d\n", card.Value)
    if card.Balak {
        fmt.Println("Kartu ini adalah BALAK ")
    } else {
        fmt.Println("Kartu ini bukan balak ")
    }
}
```

Output:



```
12 }
13
14 func main() {
15     var s1, s2 int
16
17     fmt.Print("Masukkan nilai suit pertama (0-6): ")
18     fmt.Scanln(&s1)
19     fmt.Print("Masukkan nilai suit kedua (0-6): ")
20     fmt.Scanln(&s2)
21     card := Domino{
22         Suit1: s1,
23         Suit2: s2,
24         Value: s1 + s2,
25         Balak: s1 == s2,
26     }
27
28     fmt.Printf("Kartu dibuat: [%d|%d]\n", card.Suit1, card.Suit2)
```

```
PS C:\Users\LENOVO> go run "d:\telyu\alpro_praktek\mod18\18.3.go"
Masukkan nilai suit pertama (0-6): 5
Masukkan nilai suit kedua (0-6): 4
Kartu dibuat: [5|4]
Nilai: 9
Kartu ini bukan balak
PS C:\Users\LENOVO>
```

### Deskripsi Program:

Program domino ini menggunakan tipe data Domino untuk menyimpan dua sisi kartu, nilai, dan status balak. Kartu dapat dibuat dari input user, lalu otomatis dihitung nilainya dan dicek apakah balak. Dengan cara ini, aplikasi menjadi interaktif karena pengguna bisa memasukkan variabel sendiri untuk menghasilkan kartu sesuai kebutuhan.

## Latihan 2

### Source Code:

```
package main

import (
    "fmt"
)

type Domino struct {
    Suit1 int
    Suit2 int
    Value int
    Balak bool
}

type Dominoes struct {
    Cards [28]Domino
    Count int
}

func galiKartu(d *Dominoes, target Domino)
Domino {
    var hasil Domino
    found := false
    i := 0
    for i < d.Count && !found {
        card := d.Cards[i]
        if card.Suit1 == target.Suit1 || card.Suit2
== target.Suit1 ||
        card.Suit1 == target.Suit2 || card.Suit2
== target.Suit2 {
            hasil = card
            found = true
        }
        i++
    }
    return hasil
}

func pasangKartu(a, b Domino) bool {
    return (a.Value + b.Value) == 12
}

func main() {
```

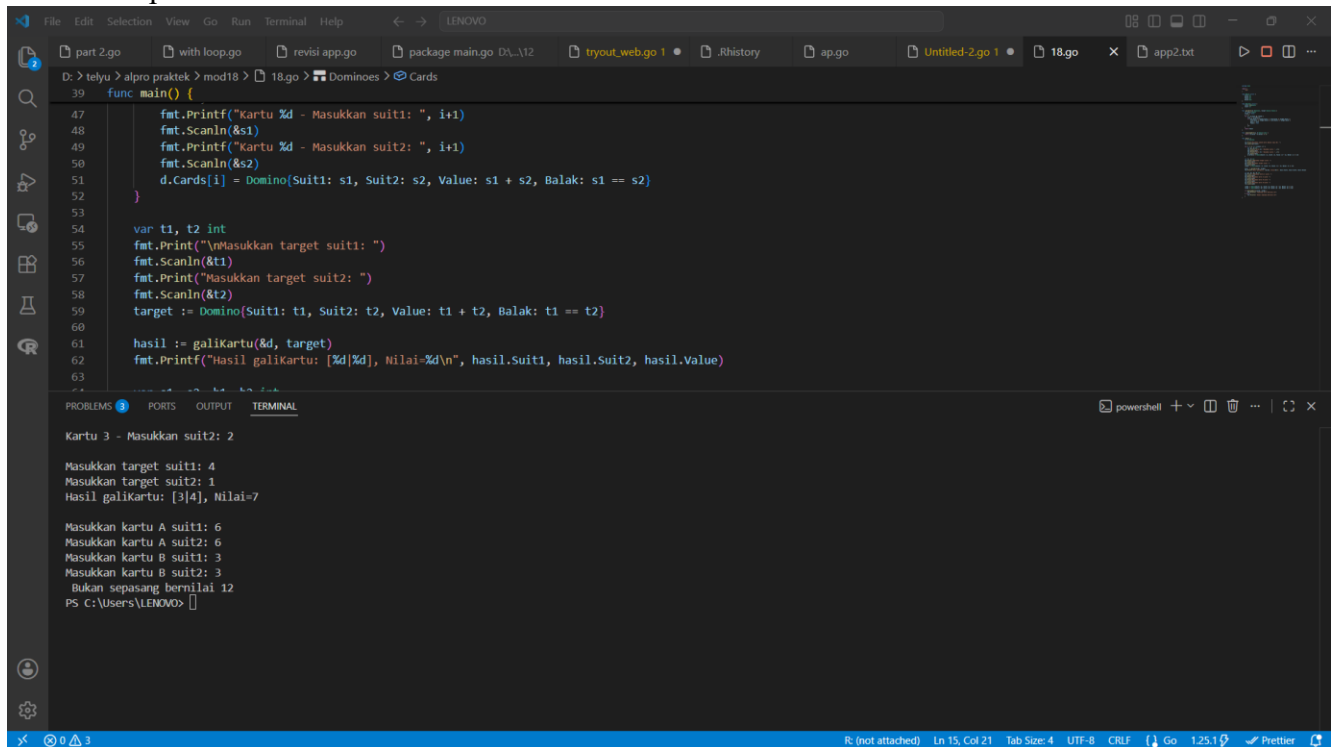
```

var d Dominoes
fmt.Print("Masukkan jumlah kartu domino
(max 28): ")
fmt.Scanln(&d.Count)

for i := 0; i < d.Count; i++ {
    var s1, s2 int
    fmt.Printf("Kartu %d - Masukkan suit1: ",
i+1)
    fmt.Scanln(&s1)
    fmt.Printf("Kartu %d - Masukkan suit2: ",
i+1)
    fmt.Scanln(&s2)
    d.Cards[i] = Domino{Suit1: s1, Suit2: s2,
Value: s1 + s2, Balak: s1 == s2}
}
var t1, t2 int
fmt.Print("\nMasukkan target suit1: ")
fmt.Scanln(&t1)
fmt.Print("Masukkan target suit2: ")
fmt.Scanln(&t2)
target := Domino{Suit1: t1, Suit2: t2, Value: t1
+ t2, Balak: t1 == t2}
hasil := galiKartu(&d, target)
fmt.Printf("Hasil galiKartu: [%d|%d],
Nilai=%d\n", hasil.Suit1, hasil.Suit2, hasil.Value)
var a1, a2, b1, b2 int
fmt.Print("\nMasukkan kartu A suit1: ")
fmt.Scanln(&a1)
fmt.Print("Masukkan kartu A suit2: ")
fmt.Scanln(&a2)
fmt.Print("Masukkan kartu B suit1: ")
fmt.Scanln(&b1)
fmt.Print("Masukkan kartu B suit2: ")
fmt.Scanln(&b2)
cardA := Domino{Suit1: a1, Suit2: a2, Value:
a1 + a2, Balak: a1 == a2}
cardB := Domino{Suit1: b1, Suit2: b2, Value:
b1 + b2, Balak: b1 == b2}
if sepasangKartu(cardA, cardB) {
    fmt.Println(" Sepasang kartu bernilai 12")
} else {
    fmt.Println(" Bukan sepasang bernilai 12")
}
}

```

## Output:



The screenshot shows a Go IDE with a code editor and a terminal. The code in the editor is a Go program that takes two suits as input and checks if they are a pair (sum of 12). The terminal shows the program's output, including prompts for suits and the result of the check.

```
func main() {  
    47     fmt.Printf("Kartu %d - Masukkan suit1: ", i+1)  
    48     fmt.Scanln(&s1)  
    49     fmt.Printf("Kartu %d - Masukkan suit2: ", i+1)  
    50     fmt.Scanln(&s2)  
    51     d.Cards[i] = Domino[Suit1: s1, Suit2: s2, Value: s1 + s2, Balak: s1 == s2]  
    52 }  
    53  
    54     var t1, t2 int  
    55     fmt.Print("\nMasukkan target suit1: ")  
    56     fmt.Scanln(&t1)  
    57     fmt.Print("Masukkan target suit2: ")  
    58     fmt.Scanln(&t2)  
    59     target := Domino[Suit1: t1, Suit2: t2, Value: t1 + t2, Balak: t1 == t2]  
    60  
    61     hasil := galiKartu(&d, target)  
    62     fmt.Printf("Hasil galiKartu: [%d|%d], Nilai-%d\n", hasil.Suit1, hasil.Suit2, hasil.Value)  
    63 }
```

Terminal Output:

```
Kartu 3 - Masukkan suit2: 2  
  
Masukkan target suit1: 4  
Masukkan target suit2: 1  
Hasil galiKartu: [3|4], Nilai=7  
  
Masukkan kartu A suit1: 6  
Masukkan kartu A suit2: 6  
Masukkan kartu B suit1: 3  
Masukkan kartu B suit2: 3  
Bukan sepasang bernilai 12  
PS C:\Users\LENOVO>
```

## Deskripsi Program:

Program domino ini memungkinkan pengguna memasukkan sendiri data kartu melalui input sehingga lebih interaktif. Prosedur galiKartu mencari kartu dalam tumpukan yang memiliki suit sama dengan kartu target yang dimasukkan. Fungsi sepasangKartu memeriksa dua kartu hasil input dan mengembalikan nilai benar jika total nilainya 12.

### Latihan 3

Code:

```
package main

import (
    "fmt"
    "strings"
)

var input string
var idx int
var currentChar byte

func start() {
    idx = 0
    currentChar = input[idx]
}

func maju() {
    idx++
    if idx < len(input) {
        currentChar = input[idx]
    }
}

func eop() bool {
    return currentChar == '!'
}

func cc() byte {
    return currentChar
}

func main() {
    fmt.Print("Masukkan untaian karakter (akhiri dengan '!'): ")
    fmt.Scanln(&input)

    if !strings.HasSuffix(input, ".") {
        input += "."
    }

    start()

    total := 0
    jumlahA := 0
    jumlahLE := 0
}
```



```
var prevChar byte = 0

for !eop() {
    ch := cc()
    total++

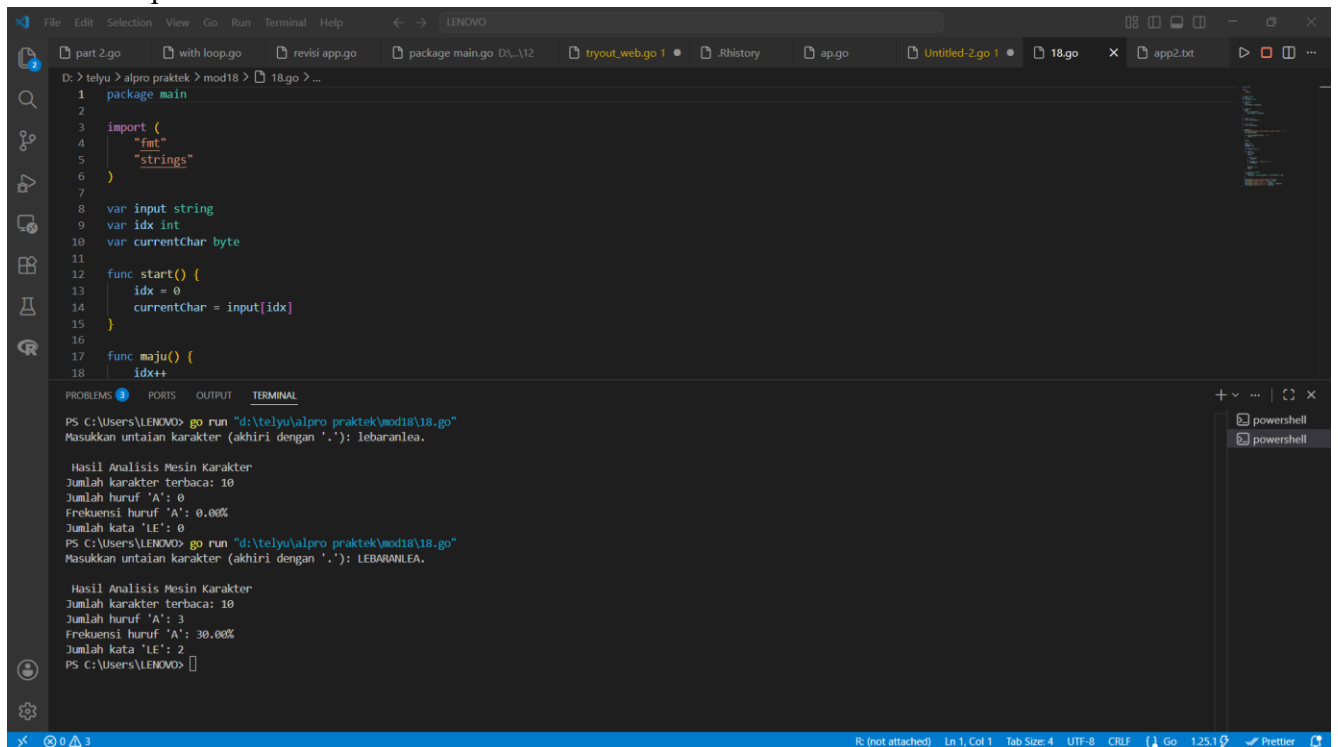
    if ch == 'A' {
        jumlahA++
    }
    if prevChar == 'L' && ch == 'E' {
        jumlahLE++
    }

    prevChar = ch
    maju()
}

var frekuensi float64
if total > 0 {
    frekuensi = float64(jumlahA) / float64(total) * 100
}

fmt.Println("\n Hasil Analisis Mesin Karakter")
fmt.Println("Jumlah karakter terbaca:", total)
fmt.Println("Jumlah huruf 'A':", jumlahA)
fmt.Printf("Frekuensi huruf 'A': %.2f%%\n", frekuensi)
fmt.Println("Jumlah kata 'LE':", jumlahLE)
}
```

## Output:



The screenshot shows a VS Code editor with a Go file named `18.go` and a terminal window. The Go code defines a simple character analysis program. The terminal shows the execution of the program twice: first with the input `lebaranlea.` and then with `LEBARANLEA.`. The output for the first run shows 10 characters, 0 'A's, 0.00% frequency for 'A', and 0 'LE' pairs. The second run shows the same 10 characters, but with 3 'A's (30.00% frequency) and 2 'LE' pairs.

```
1 package main
2
3 import (
4     "fmt"
5     "strings"
6 )
7
8 var input string
9 var idx int
10 var currentChar byte
11
12 func start() {
13     idx = 0
14     currentChar = input[idx]
15 }
16
17 func maju() {
18     idx++
19 }
```

PS C:\Users\LENOVO> go run "d:\telyu\alpro praktek\mod18\18.go"

Masukkan untaian karakter (akhiri dengan '.'): lebaranlea.

Hasil Analisis Mesin Karakter

Jumlah karakter terbaca: 10

Jumlah huruf 'A': 0

Frekuensi huruf 'A': 0.00%

Jumlah kata 'LE': 0

PS C:\Users\LENOVO> go run "d:\telyu\alpro praktek\mod18\18.go"

Masukkan untaian karakter (akhiri dengan '.'): LEBARANLEA.

Hasil Analisis Mesin Karakter

Jumlah karakter terbaca: 10

Jumlah huruf 'A': 3

Frekuensi huruf 'A': 30.00%

Jumlah kata 'LE': 2

PS C:\Users\LENOVO>

## Deskripsi:

Program Mesin Abstrak ini bekerja dengan cara membaca string huruf satu per satu sampai tanda titik sebagai akhir. Fungsi `start`, `maju`, `eop`, dan `cc` dipakai untuk mengatur posisi pembacaan karakter. Dengan langkah sederhana ini, pemula bisa menghitung jumlah huruf, mencari berapa kali huruf A muncul, dan mendeteksi pasangan huruf "LE" tanpa perlu logika yang rumit.

## DAFTAR PUSTAKA

- Dasa, Rinaldi Munir. *Modul Praktikum Algoritma dan Pemrograman 2*. Telkom University, 2024.
- Cormen, Thomas H., et al. *Introduction to Algorithms*. MIT Press, 2009.
- Wirth, Niklaus. *Algorithms + Data Structures = Programs*. Prentice Hall, 1976.
- Knuth, Donald E. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley, 1997.
- Sedgewick, Robert, dan Wayne, Kevin. *Algorithms*. Addison-Wesley, 2011.